Assignment - 1

Aim:

To study and understand client server architecture, distributed communication, using 2 paradigms: low level TCP sockets (and) high level gRPC (Remote Process Call framework).

Objectives:

- Understand the fundamental principles of client-server architecture model in distributed systems.
- Implement bidirectional network communication using Berkely TCP sockets (SOCK_STREAM).
- Understand network adressing using IP addresses (layer 3) and Port Number (layer 4).
- Implement Remote Procedural Call (RPC) service using gRPC & Protocol Buffers (ProtoBuf) over HTTP/2.

~~Tho~~

Theory:

1) Client-server Architecture:

The Client-server Architecture is a centralized computing model dividing tasks ~~and~~ or workloads between service providers (servers) and service Requesters (Clients):

- Servers: A persistently running entity listening ~~for~~ for incoming requests across a network, executing computation or resource lookup & returning responses.
- Client: An active entity that initiates communication by establishing a conn^n, submitting ~~req~~ request payloads & awaiting results.

Assignment - 1

Aim:

To study and understand client-server architecture, distributed communication, using 2 paradigms: low-level TCP sockets and high level gRPCs (Remote Process Call framework).

Objectives:

- Understand the fundamental principles of Client-server architecture model in distributed systems.
- Implement bidirectional network communication using Barkely TCP sockets (SOCK_STREAM).
- Understand network adressing using IP addresses (Layer 3) and Port Number (Layer 4).
- Implement Remote Procedural Call (RPC) service using GRPC & Protocol Buffers (ProtoBuf) over HTTP/2.

~~The~~

Theory:

1) Client-server Architecture:

The Client-server Architecture is a centralized computing model dividing tasks ~~and~~ or workloads between service providers (servers) and service Requesters (Clients).

- Servers: A persistently running entity listening ~~for~~ for incoming requests across a network, executing computation or resource lookup & returning responses.
- Client: An active entity that initiates communication by establishing a conn^n, submitting ~~req~~ request payloads & awaiting results.

PCCOE

- Characteristics: Asymmetric relationship, centralized resource management, loose coupling over network protocols and scalable horizontally or vertically.

2) Distributed Communication:

Enables autonomous processes running in distinct memory spaces - either on the same OS or across physical hardware seperated by networks - to coordinate & exchange information. Distributed comm^n bridges differences in:

- Heterogeneity: Dissimilar hardware architecture (x86 vs ARM) & byte ordering (Big-endian vs Little Endian).
- OS: Windows, Linux, & macOS inter-communication.
- Network Latency & Reliability: Managing network partitions, packet loss & transmission delays.

3) Roles of Client & Server:

Aspect	Client Role	Server Role
Lifecycle	~~Eph~~ Ephemeral (Started on user action, exists when done)	Persistant demon (runs indefinitely in bkg)
Initiation	Active Initiator	Passive listener
Addressing	Needs to know Servers IP address & Port	Binds a well known Port on host Interfaces
Concurency	Usually single session or user facing thread	~~Mutti~~ Multi-threaded/ Event driven to serve concurrent clients.

4) Socket Communication & Internalworking:

- A socket is an endpoint abstraction provided by the OS for sending & receiving data over computer network. In UNIX/ POSIX and windows(Winsock), it is treated as an I/O handle

- Internal Socket Lifecycle:

SERVER	CLIENT
1) Socket(AF_INET, SOCK_STREAM)	1) socket(AF_INET, SOCK_STREAM)
2) bind(IP, PORT)	
3) Listen(BACKLOG)	
4) accept() ← SYN handshake (TCP 3-Way)	2. Connect(SERVER_IP, PORT)
[spawns worker Thread]	
5) recv() ← Request Data	3. send()
6) send() → Response Data	4. recv()
7) close() ← FIN/ACK	5. close()

5) IP addres & Port Number:

Network communication relies on two-tiered addressing:

- IP addressing (Network layer - Layer 3): Uniquely identifies a physical or virtual machine interface on an IP network (eg: 127.0.0.1 for loopback, 192.168.1.15 for private LAN).
- Port Number (Transport layer - Layer 4): A 16 bit Integer (Ranging $0 of to 65535) that identifies specific application process running on host machine:
  - i) well known port ($0 - 1023$): Reserved for core services
  - ii) Registered / Ephemeral Ports ($1024 - 65535$): Custom Applications

PCCOE

6) TCP vs UDP communication

Feature ~~Connection Type~~	TCP (Transmision Control Protocol)	UDP (User Datagram Protocol)
• Connection Type	• Connection-Oriented	• Connectionless
• Data transfer Unit	• Continuous Byte stream	• Discreate datagram packets
• Reliability	• Guaranteed delivery	• Best effort
• Ordering	• In-order sequencing guaranteed	• No sequence preservation
• Flow & Congestion Control	• Yes	• No flow or Congestion control
• Overhead	• Higher header overhead	• minimal overhead

7)

Parameter	Barkely sockets (Low level)	gRPC (High level RPC)
• Abstraction	• Low Level	• High level
• Protocol Found^n	• Transport Layer	• Application layer
• Data Serializ^n	• Manual Encoding/ decoding	• Automatic, Strongly-typed binary protocol Buffer
• Contract Enforcement	• Informal Eadhoc application logic	• Strict API contract via .proto schema
• Performance	• Minimal protocol overhead, maximum Control	• Near Socket speeds due to binary protobuff HTTP/2

PCCOE

Conclusion:

In this assignment, both low level socket communication & modern Remote Procedural Call (gRPC) paradigms were successfully implemented & analyzed:

- TCP sockets demonstrated mechanics of byte streams, OS level socket system calls (bind, listen, accept), & thread-based concurrent client management.
- gRPC abstracted the transport complexities using Protocol Buffers & HTTP/2, enabling strongly-typed remote method invocation with automatic serialization & multi platform compatibility.

PCCOE

Assignment-2

Aim:

To design, simulate & analyze distributed coordinator/ leader election algorithms using Bully algorithm & Ring algorithm, modelling node crashes, dynamic failure detection, message exchanges & coordinator reelection upon node recovery.

Objectives:

1) Understand the fundamental role and necessity of a Coordinator / Leader in distributed systems.
2) Analyze impact of coordinator failure & understand how remaining nodes detect failures
3) Implement & simulate Bully election Algorithm
4) Implement and simulate Ring election algorithm
5) Track and evaluate election message workflows
6) Handle dynamic nodefailure, crash bypass and Recovered node reelection.

Theory:

1) Coordinator/ Leader Election in Distributed system:

In distributed systems, autonomous processes frequently need to coordinate their actions. Rather than having all nodes make duplicate decisions or contend simultaneously, systems designate a single distinguished process known as Coordinator (or Leader)

2) Coordinator and Process failure:

- Failure Model: we assume Crash-stop (fail stop) model where processes may halt unexpectedly without exhibiting Byzantine behaviour.
- Failure Detection: When regular node requests service from coordinator & receives no response within a specified time Interval, it assumes coordinator has failed & triggers an election.
- Network assumptions: Communication channels are Reliable and messages between non-faulty nodes are delivered in sequence.

3) The Bully algorithm:

- Operates under the assumption that every process posseses a unique numerical Process ID. A process with Higher PID always has higher priority & 'bullies' processes with lower PIDs into submission
- Algorithm:

1> Detection and Initiation: Process notices the coordinator is unresponsive. It initiates an election by sending an election message to all processes with higher IDs

2> Response (OK / ANSWER):

- If any higher process is alive, it replies with an OK (or ANSWER) message to P_k.
- On receiving OK, P_k's role in leading the election ends; it steps aside and waits for higher process to finish election.

PCCOE

3) Winning Election:
If process receives no Ok messages within timeout period, P_k wins election.

4) Coordinator Announcement:
The winning Process broadcasts a COORDINATOR announcement to all active lower ranking processes

5) Recovery of failed Process:
When a crashed procer recovers, it immediately holds an election. If it holds highest PID in entire system, it bypasses election inquiry & directly broadcasts COORDINATOR, immediately reclaiming leadership from any ~~other~~ lower-ranked interim leader.

4) The Ring Algorithm:

- Arranges allprocesses into logical, ordered unidirectional ring.
- Each process only needs to know the network address of its immediate successor in Ring.
- Algorithm:

a) Initiation: when any active process notices the coordinator has crashed, it creates Election message containing an active candidate list Initialized with its own ID.

PCCOE

b) Forwarding Around the Ring:
- P_k transmits this message to its immediate successor in the ring.
- If immediate successor is crashed, P_k skips it & forwards to next active successor.

c) Candidate List Accumulation:
- When active process P_j receives $\text{ELECTION}(\text{list})$:
- It appends its own ID to candidate list
- It forwards the updated message to its active successor in the ring.

d) Loop completion & leader Election:
When election message completes a full circuit & Returns to original initiator:
The Initiator evaluates final list.

5) Handling failed Processes:
- During Election in Bully Algorithm:
- If a higher node crashes midway through taking over election, the initial sender's Response timer expires without receiving subsequent COORDINATOR messages, prompting timeout-driven Reelection.

- During Election in Ring Algorithm:
- Node crashes are handled through fault-tolerant successor skipping. If node P_i's successor P_{i+1} does not acknowledge transmission, P_i accesses its topology table & forwards the msg to $P_{(i+2)}$, maintaining Ring integrity.

Conclusion:

In this assignment, distributed coordinator election was implemented & analyzed using both Bully and Ring algorithms:

1) The Bully Algorithm prioritizes rapid convergence using parallel broadcasts & hierarchical authority.
2) The Ring algorithm maintains a predictable $O(N)$ message upper bound & minimizes link requirements, but introduces higher sequential latency as election tokens travel node by node.

Assignment -3

Aim:

To design, implement & analyze a deadlock detection and Resolution mechanism in distributed systems using:

1) A Wait-For-Graph (WFG) with Depth-First Search (DFS) & color cycle detection
2) The Chandy-Misbra-Haas (CMH) probe based distributed edge-chasing algorithm.
3) Automated Deadlock Recovery using customizable victim selection policies.

Objectives:

1) Understand causes & necessary conditions of Deadlock in concurrent and distributed systems.
2) Formulate process-resource dependencies by translating a Resource Allocation Graph (RAG) into a Wait For Graph [WFG].
3] Implement directed graph Cycle detection using DFS with 3 state coloring (WHITE, GRAY, BLACK) to extract elementary cycles
4) Implement Chandy-Misra-Haas (CMH) edge chasing algorithm for distributed environments lacking centralized graph visibility.
5) Implement dynamic Deadlock Recovery strategies.

Theory:

1) Deadlock:

- In distributed system is a state where a set of concurrent proceses is permanently blocked because each process holds atleast one shared Resource & is waiting to acquired another resource held by another procer in same set.
- Since all involved processes are blocked waiting for events that only other blocked proceses can trigger, none can make progress without external intervention.

2) Causes & Conditions of Deadlock (Coffman Condⁿ):

→ Condition	→ Description	→ Distributed Conte
- ~~For~~ Mutual Exclusion	- Resources are non shareable; only one Process can hold Resource at given time	- Distributed lock, database write locks, Exclusive File handles
- Hold & wait	- A process holds currently allocated Resources while Requesting additional Resource	- Transaction holds lock 'A' on Node 1 & Requests lock 'B' on Node 2.
- No preemption	Resources cant be forcibly seized; they can only be Released voluntarily by holding process	- Locks cant be Revoked without aborting or Rolling back the transaction.

PCCOE

Condition	Description	Distributed Context
- Circular wait	- A closed chain of processes exist: P_0 of wait for P_1, P_1 for P_2 & so on to P_0	- Circular dependency across processes situated on seperate physical machines

3) Representation of Process Dependencies : RAG & WFG :

→ Resource Allocation Graph (RAG) :

- A bipartite directed graph $G = (V, E)$ where the vertex set V consists of 2 disjoint sets:
- Processes : $P = (P_1, P_2, \dots, P_n)$
- Resources : $R = (R_1, R_2, \dots, R_m)$
- Edges :
  - Allocation edge ($R_j \to P_i$) : Resource R_j is currently held by process P_i.
  - Request edge ($P_i \to R_j$) : Process P_i is blocked waiting for Resource R_j.

→ Wait-For-Graph (WFG) :

- Collapses Resources out of RAG to represent direct process to process dependencies.
- Vertices - Processes only ($V = P$)
- Directed Edge ($P_i \to P_j$) : Process P_i is waiting for Resource

PCCOE

4) Cycle Detection for identifying Deadlock (DFS 3-State Coloring).

— To detect elementary cycles in directed graphs without entering infinite loops, nodes are classified into 3 color states during Depth-First search:-

- WHITE (Unvisited): The process has not yet been encountered
- GRAY (Visiting): The Process is in active DFS Recursion call stack.
- BLACK (Visited): The Process & all its downstream dependencies have been completely explored.

★ Logic for cycle Detection:

1) Traverse all nodes in graph starting with color WHITE.
2) When visiting node u, set color[u] = GRAY & push ~~out~~ u onto Current path stack.
3) For each neighbour v of u of u ($u \to v$):
   - If color[v] == GRAY: A back-edge is found! A circular wait cycle exists from v through u back to v. The cycle is recorded.
   - If color[v] = WHITE: Recursively call DFS on v.
4) pop u from Path stack & mark color[u] = BLACK.

FCCOE

5) Deadlock Recovery :

- Once cycle is identified, system must Recover by by breaking circular wait chain:

a) Victim selection : Choosing which process in cycle to terminate or roll back based on a policy.
- LOWEST_PRIORITY : Selects the Process with lowest assigned priority, shielding critical system tasks.
- FEWEST - RESOURCES : Selects the process holding the minimum number of Resources to minimize rollback & compensation costs.
- HIGHEST_PID / lowest_PID : Deterministic numeric selection to break ties consistently across nodes.

b) Process Abort / Preemption :
- The victim process is aborted (status = ABORTED).
- All resources held by victim are Released.
- Dependent processes waiting on victim are unblocked (status - ACTIVE)

c) Re-evaluation :
The detection algorithm runs again to confirm that all overlapping or secondary cycles have been Resolved.

LCCOE

Conclusion:

- In this assignment, deadlock detection & Recovery were implemented and analyzed across both centralized & distributed environments.
- Wait-For Graph (WFG) modelling: Demonstrates that collapsing process Resource dependencies into directed graphs allows cycle detection via DFS 3-state coloring with O(V+E) Time complexity.
- Chandy-misra-Haas Algorithm: Solves distributed detection problems without a centralized coordinator by routing lightweight probe messages along dependency edges with O(E) message overhead.